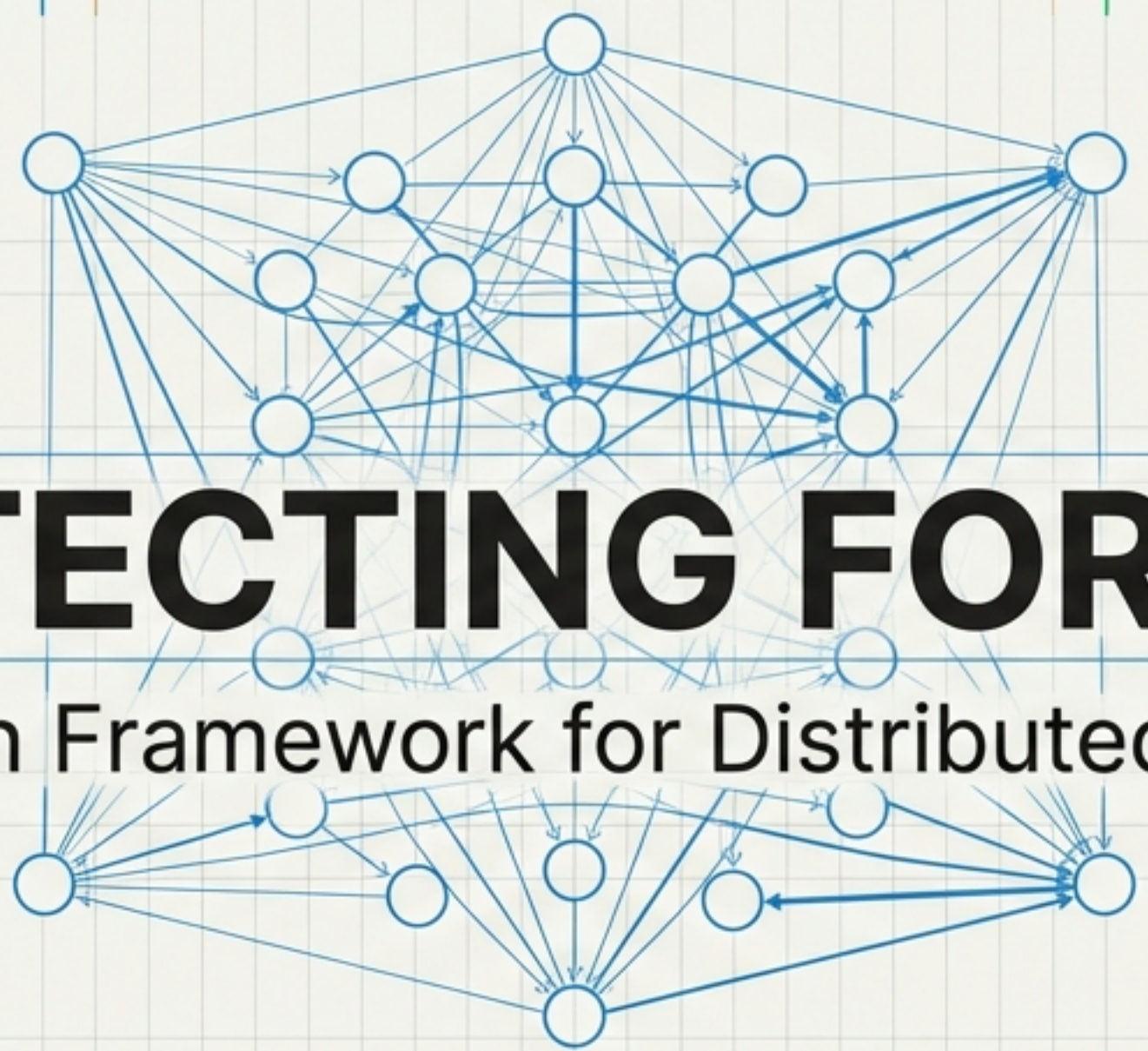




ARCHITECTING FOR SCALE

A Decision Framework for Distributed Systems

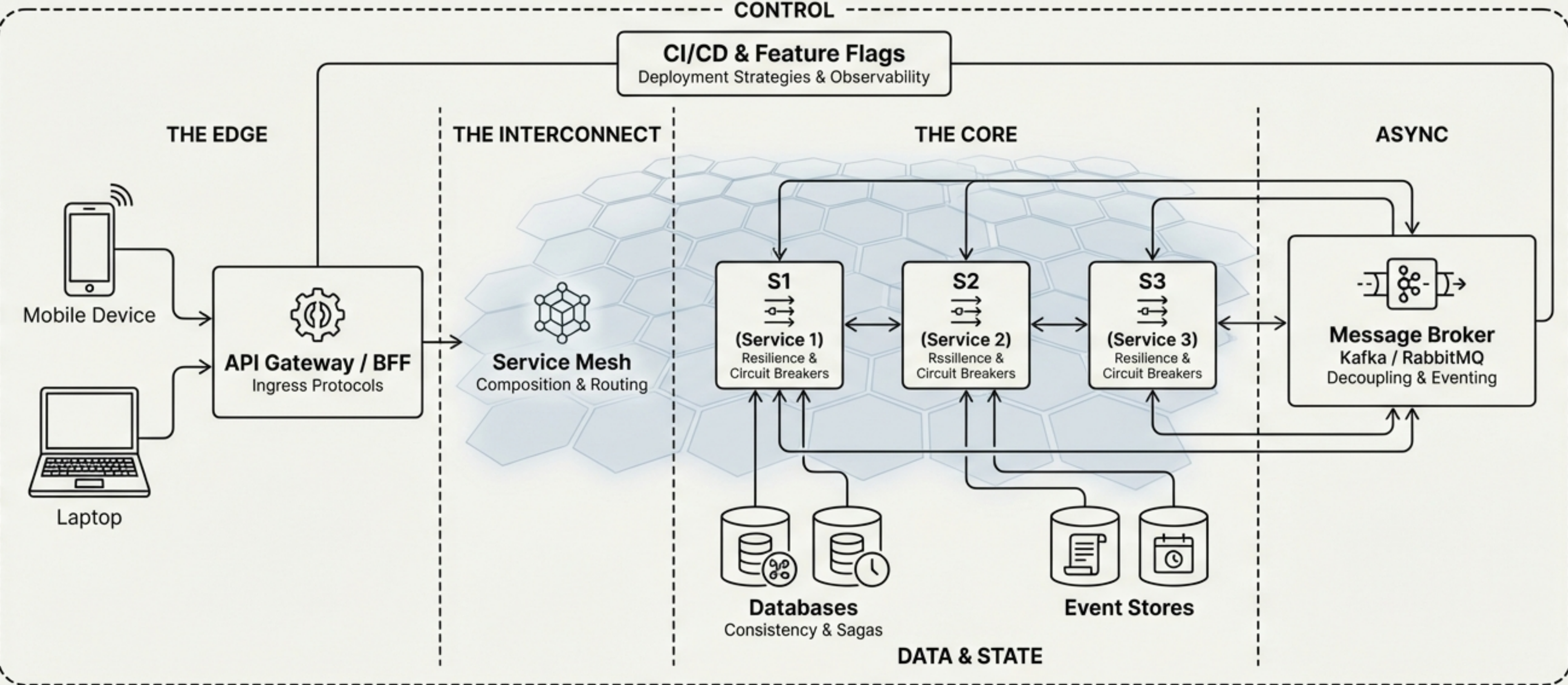
VERSION 1.0

TARGET: SYSTEM ARCHITECTS /
ENGINEERING LEADS

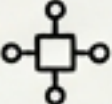
SCOPE: COMMUNICATION, DATA,
RESILIENCE, DEPLOYMENT

A comprehensive reference guide for the 7 critical domains of distributed architecture.

The Distributed Landscape



API STYLES:
Ingress Protocols



GATEWAYS:
Composition & Routing



RESILIENCE:
Circuit Breakers & Limits



DATA PATTERNS:
Consistency & Sagas



EVOLUTION:
Deployment Strategies

API Communication Styles

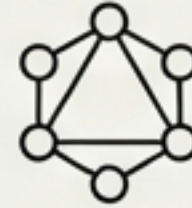


REST

Best for: Public APIs, CRUD, Caching.

Pros: Universal support, HTTP Caching, Mature tooling.

Cons: Over/Under-fetching, Chatty.

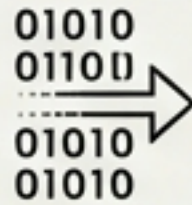


GraphQL

Best for: Complex data, Mobile, Flexible Queries.

Pros: Precise fetching, Single Endpoint, Strong typing.

Cons: Caching complexity, N+1 query risks.

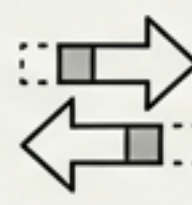


gRPC

Best for: Internal Microservices, Low Latency.

Pros: High perf (Protobuf), Streaming, Type safety.

Cons: Browser support requires proxy, Binary illegibility.



WebSockets

Best for: Real-time, Bidirectional, Gaming/Chat.

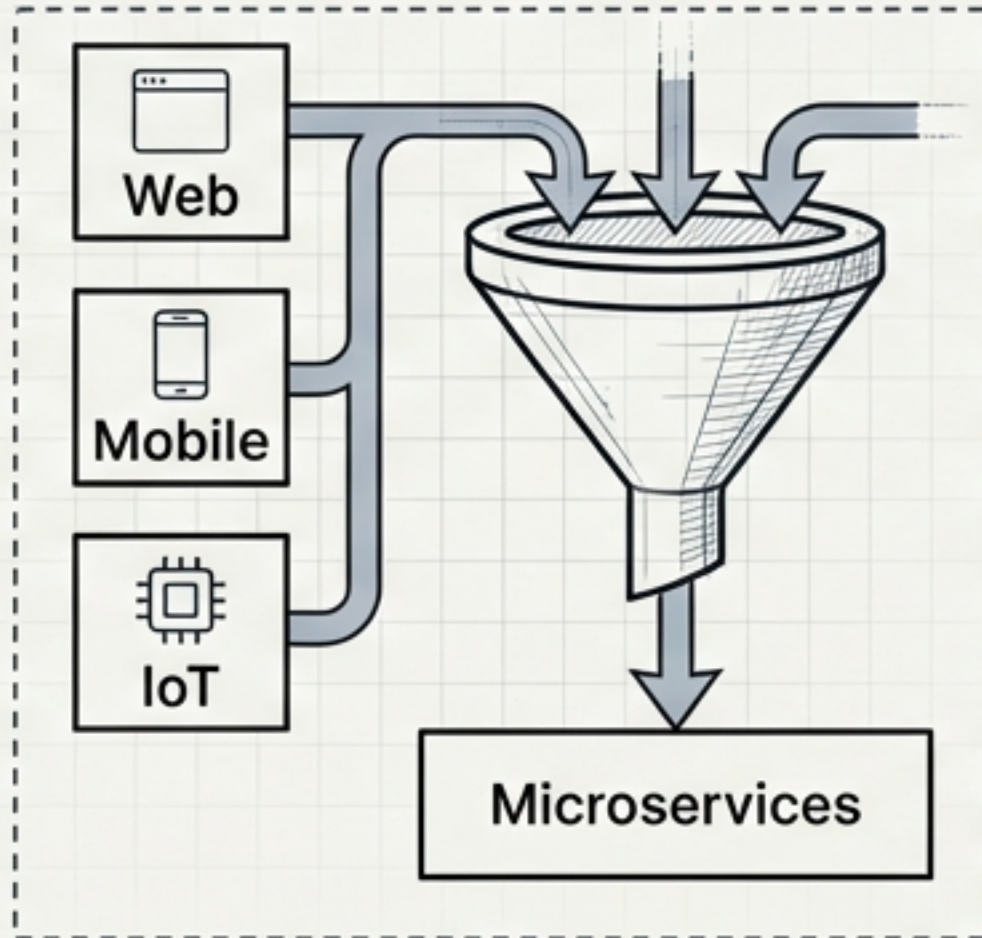
Pros: Full-duplex, Low latency, Stateful.

Cons: Hard to load balance, Stateful connection.

DECISION GUIDE: Choose REST for public reach. Choose gRPC for internal speed. Choose GraphQL for flexible clients.

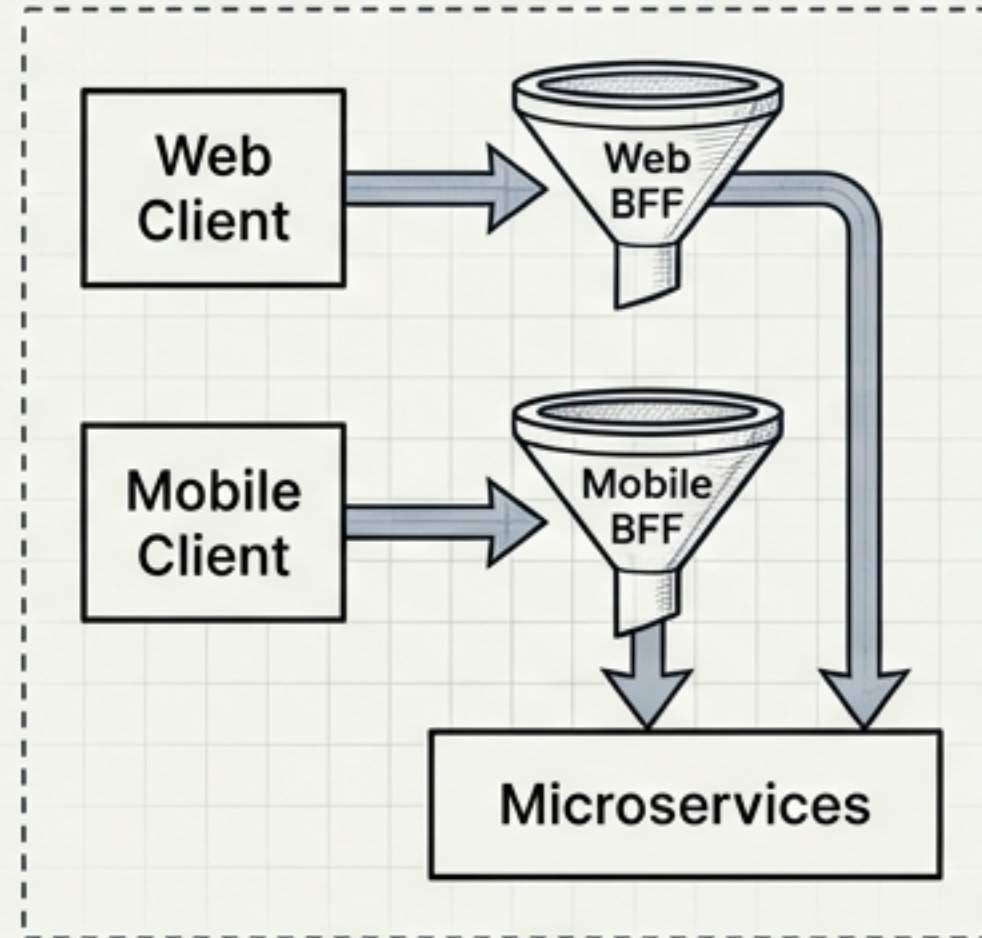
Entry Points and Composition

API Gateway



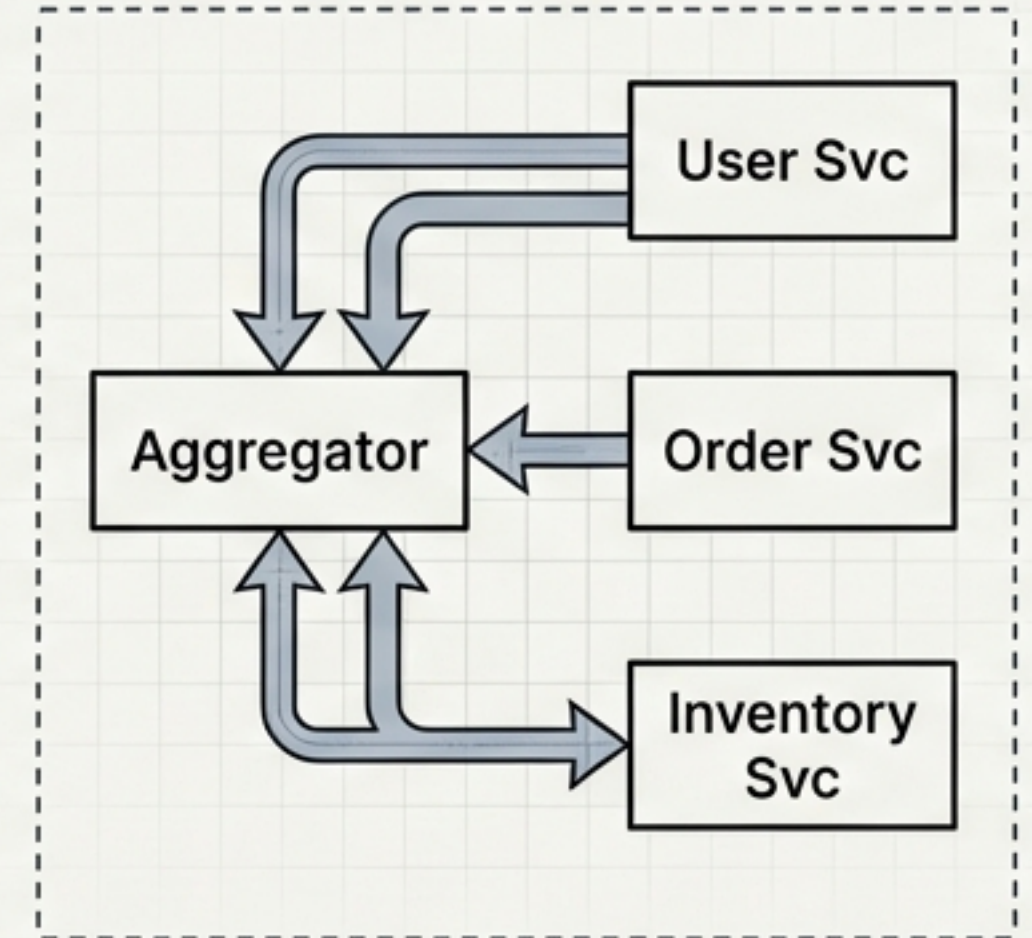
Single Entry Point.
Centralized Auth & Rate Limiting.

Backend for Frontend (BFF)



Client-Specific Optimization.
Parallel development.

Aggregator Pattern



Composite Response.
Reduces client round-trips.



Production Note: Common combination is **Gateway (Security/Throttling)** -> **BFF (Client Logic)** -> **Aggregator (Data Composition)**.

Asynchronous Messaging & Decoupling

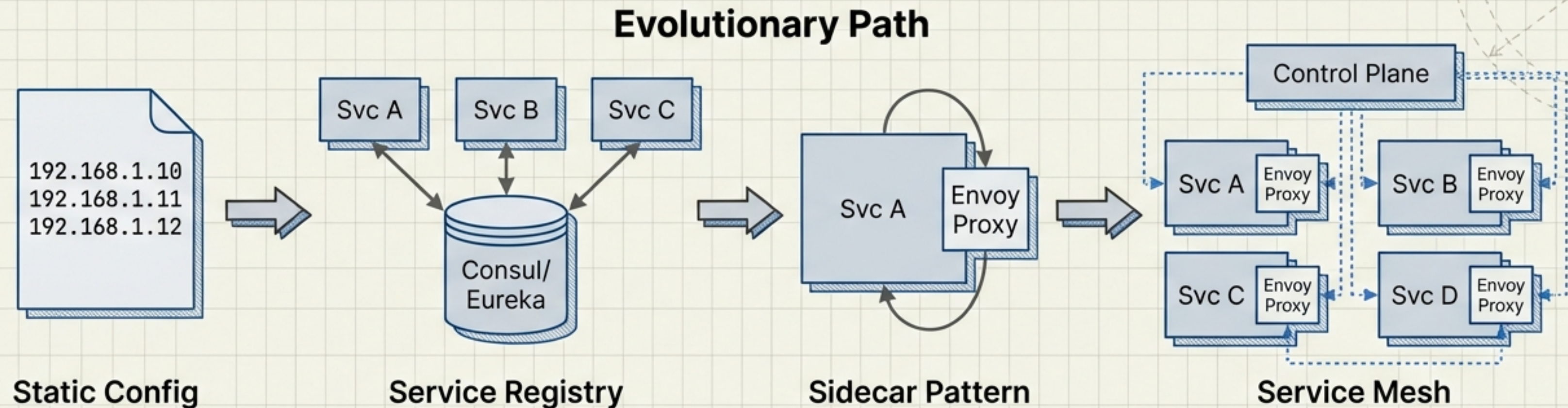
Pattern	Delivery Model	Coupling	Best For
Pub/Sub (Kafka/SNS)	Fan-out (One-to-Many)	Very Loose	Event notifications, Cache invalidation
Message Queue (SQS/RabbitMQ)	Load Leveling (One-to-One)	Loose	Expensive background jobs, Work distribution
Event-Driven	Reactive State	Decoupled	Complex workflows across domains

Delivery Guarantees Spectrum



“Use **Queues** to flatten load spikes; use **Pub/Sub** to decouple domains.”

Service Discovery & The Mesh



Complexity vs. Value

Registry	Low Complexity	Basic Discovery
Sidecar	Medium Complexity	TLS/Logging offload
Mesh	High Complexity	Zero-Trust mTLS, Traffic Splitting, Observability

ADOPT A MESH ONLY WHEN OPERATIONAL COMPLEXITY OF SERVICES > COMPLEXITY OF MANAGING THE MESH (Typically 10+ services).

Distributed Transactions & Consistency

Trade-off Slider

Strong Consistency (ACID)

High Availability & Scale



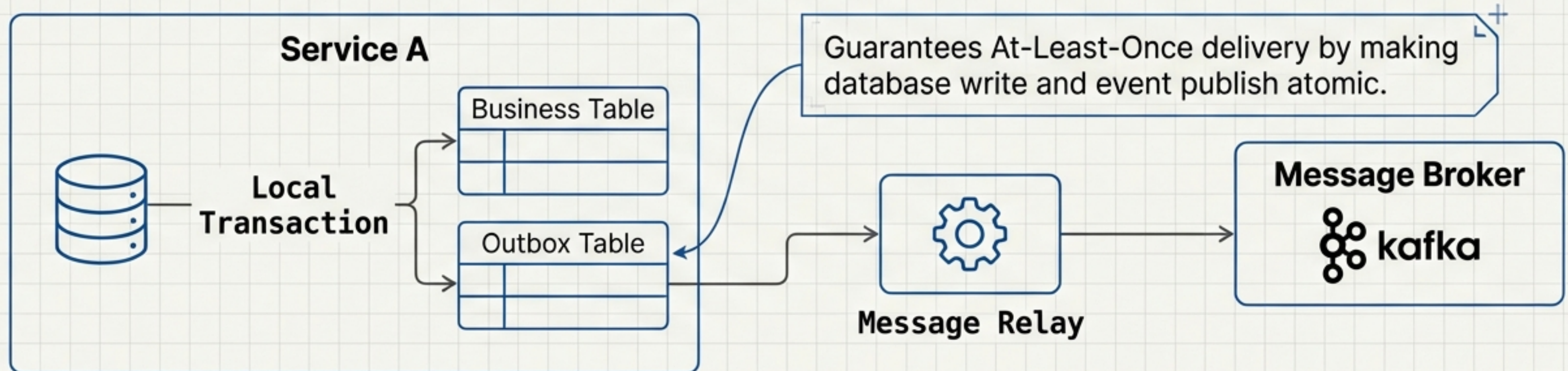
Two-Phase Commit (2PC)

Blocks resources.
Does not scale.
Use for: Banking.

Saga Pattern

Eventual Consistency.
Complex Rollbacks.
Use for: E-commerce.

The Dual-Write Solution: Outbox Pattern



Advanced Data Management

The Perfect Match:

Events are the Source of Truth → CQRS → projections build the Read Views.

CQRS

Command Side (Write)

Optimized for logic/validation.
High throughput.

Query Side (Read)

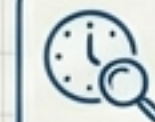
Optimized for retrieval.
Denormalized views.



Independent
Scaling.

Event Sourcing

State as a sequence of events

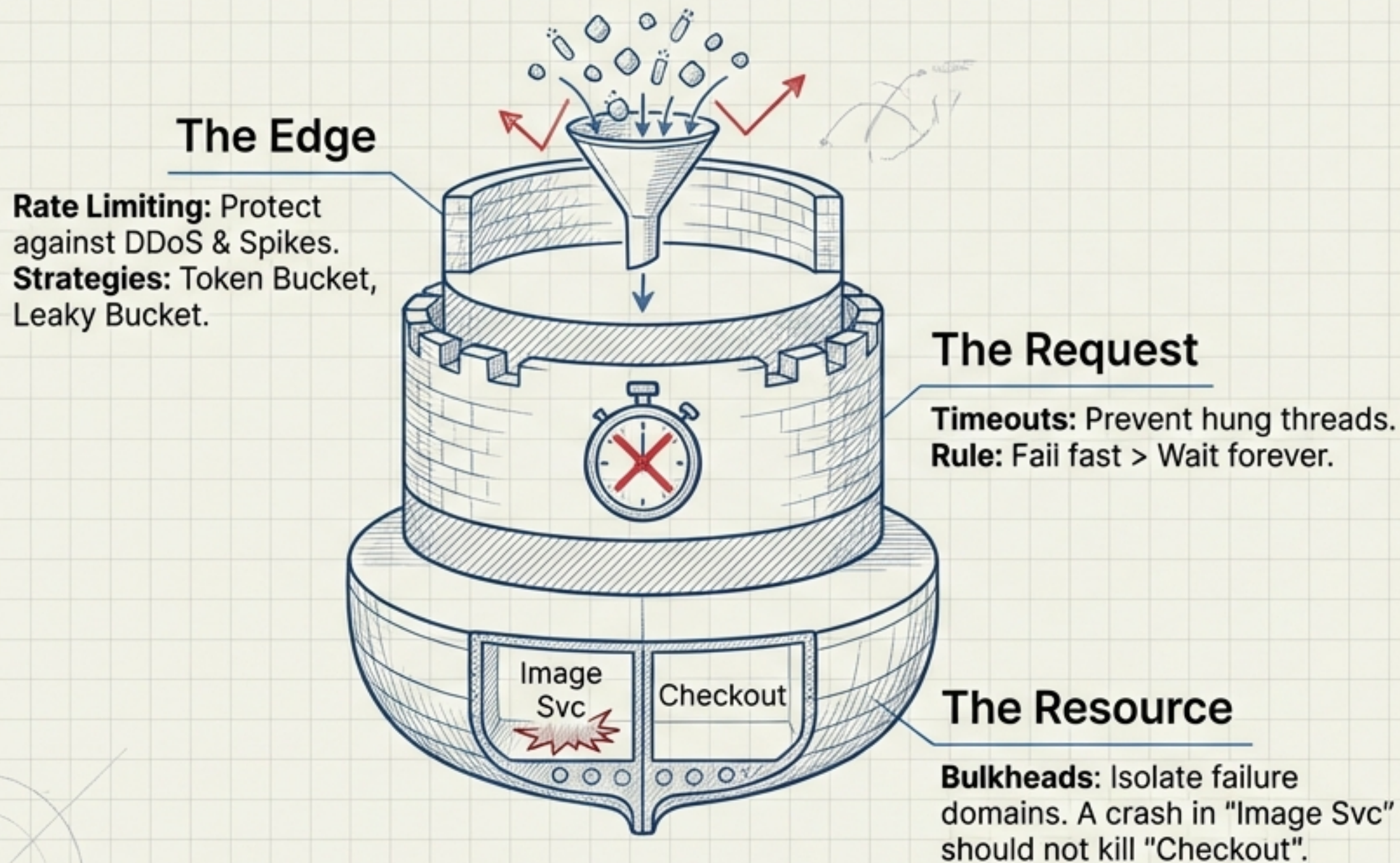


Complete Audit Trail
& Time Travel.

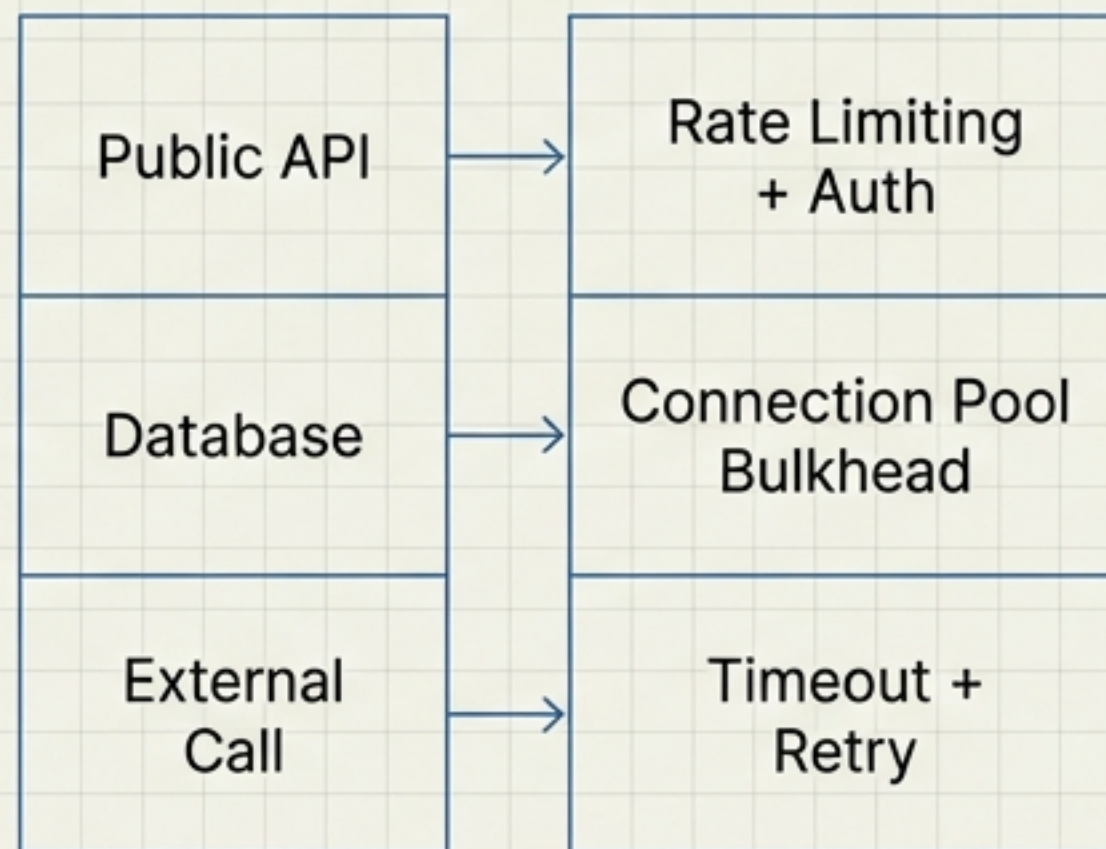
CAP Theorem Context: These patterns favor Availability and Partition Tolerance (AP) over immediate Consistency.

Defensive Resilience: Shielding the System

Resilience Stack

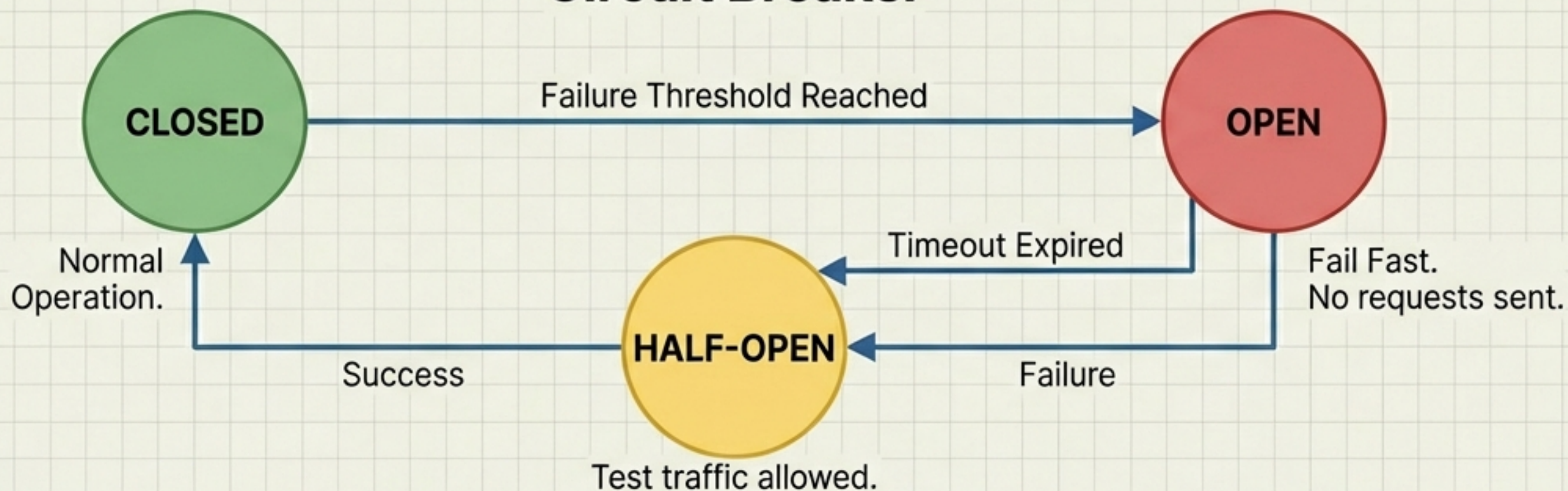


Use Case Matrix

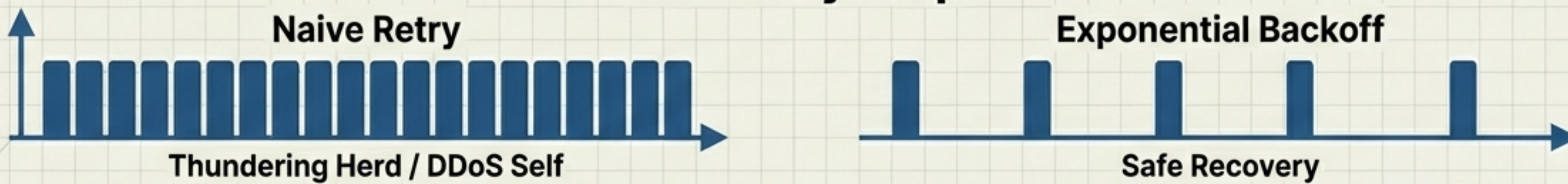


Reactive Resilience: Recovering from Failure

Circuit Breaker

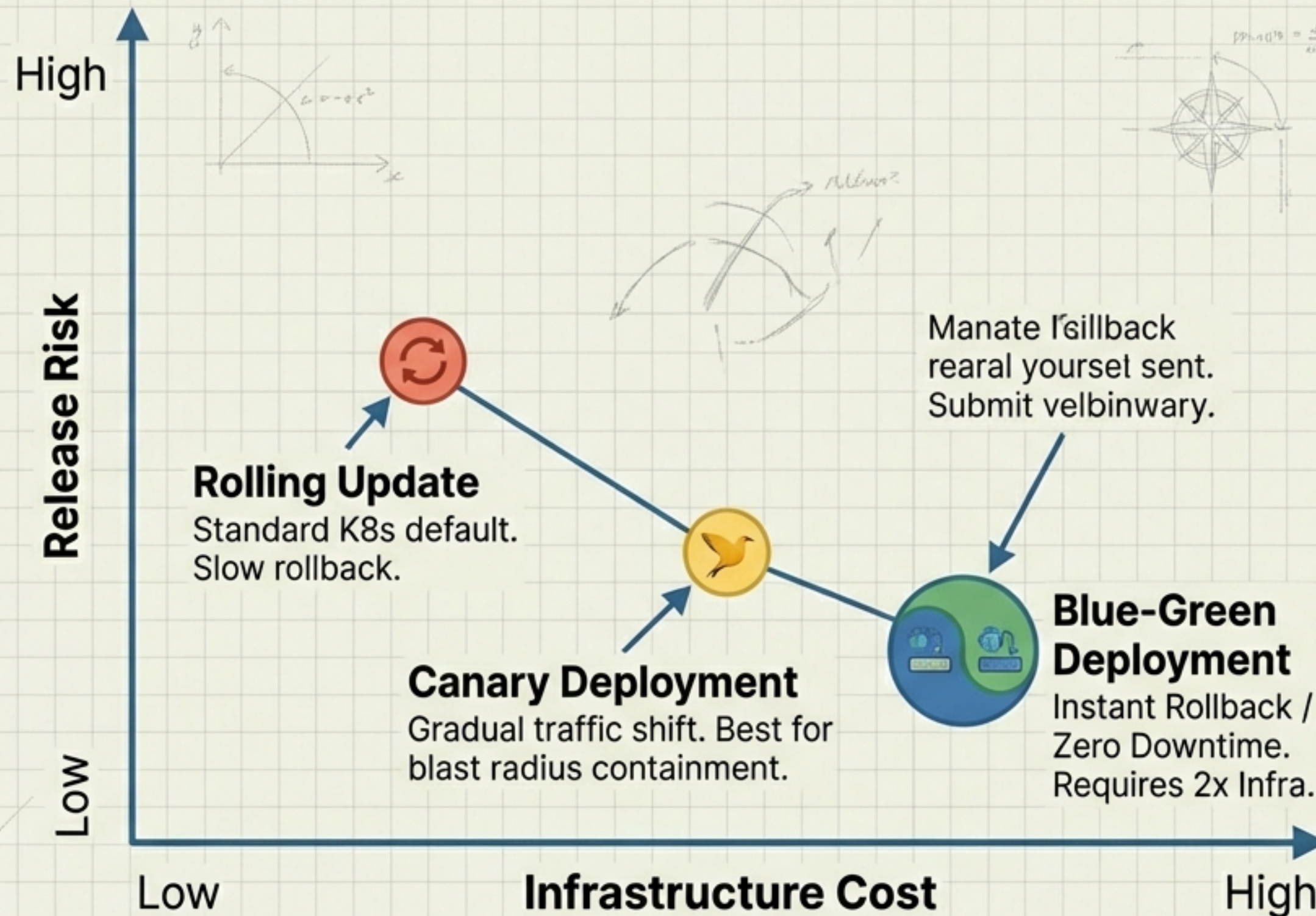


The Retry Trap



Golden Rule: Never use Retry without a Circuit Breaker.

Deployment Strategies



Canary Signals

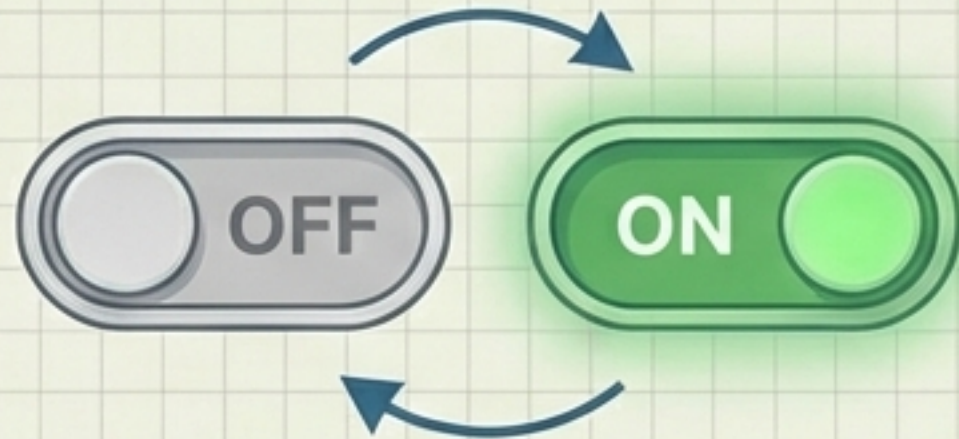
Metrics to watch during rollout:

- Error Rate (HTTP 5xx)
- Latency (p99)
- Business conversion rate

Evolution & Control

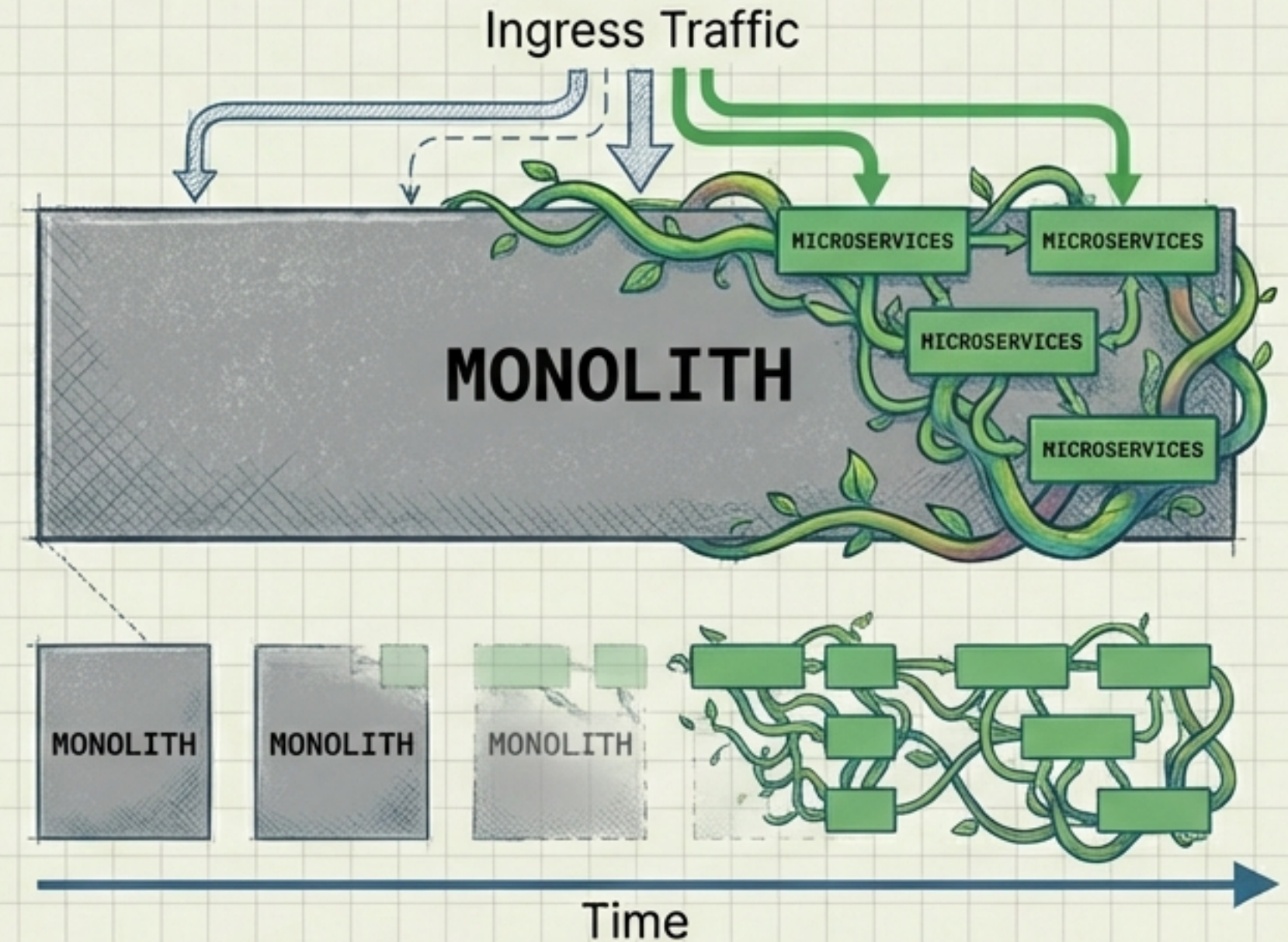
Pattern 1: Feature Flags

Decouple Deploy (Ops) from Release (Business)



Release Toggles (WIP)
Ops Toggles (Kill Switches)
Experiment Toggles (A/B)

Pattern 2: Strangler Fig Migration



Don't rewrite. Intercept and replace functionality endpoint by endpoint.

The Master Cheatsheet

SCENARIO	PRIMARY PATTERNS	RATIONALE
E-commerce Checkout	Saga + Outbox	Distributed transaction across Order/Pay/Inventory without blocking.
Financial Audit System	Event Sourcing + CQRS	Perfect historical audit trail. Temporal queries.
Public API Gateway	REST + Rate Limiting	Universal compatibility and protection from abuse.
Internal High-Freq Svc	gRPC + Service Mesh	Low latency, strong typing, zero-trust security.
Real-time Dashboard	WebSockets + Pub/Sub	Bidirectional live data stream.
Legacy Migration	Strangler Fig + Flags	Incremental replacement with safety toggles.
Global Reporting	CQRS (Read Replicas)	Isolate heavy read loads from write logic.

Real-World Architecture Hall of Fame

NETFLIX

Key Contribution: Resilience Engineering.

Patterns:

- Hystrix (Circuit Breaker)
- Zuul (Gateway)
- Chaos Monkey

UBER

Key Contribution: Scale & Coordination.

Patterns:

- Saga (Trip workflows)
- Ringpop (Service Discovery)

AMAZON

Key Contribution: Deployment Velocity.

Patterns:

- Blue-Green Deployment
- 2PC (Internal)
- Outbox

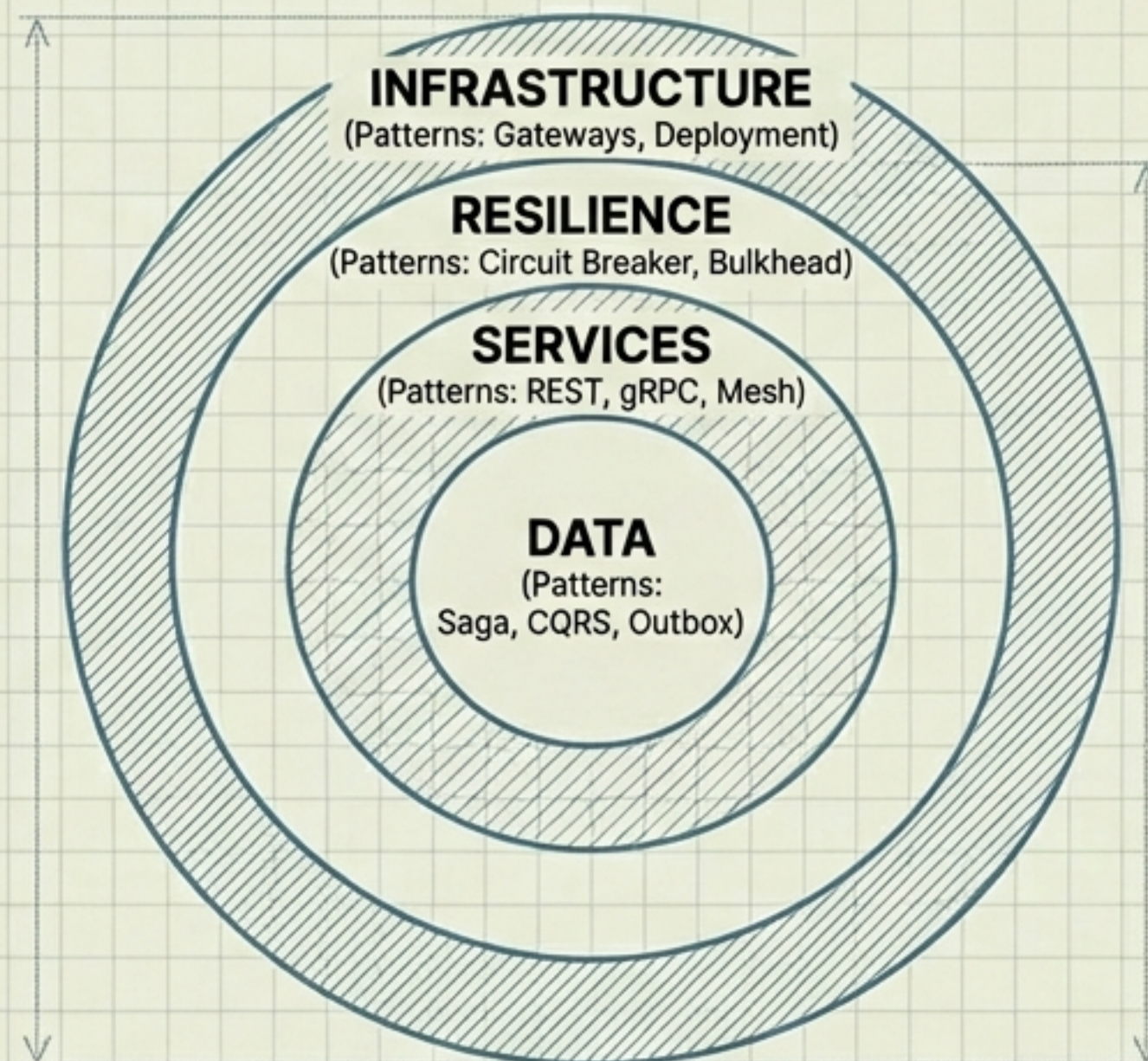
LINKEDIN

Key Contribution: Streaming Data.

Patterns:

- Kafka (Pub/Sub)
- Event Sourcing

The Architectural Tapestry



**“There are no best practices, only trade-offs.
The goal is the management of complexity.”**

Recommended Reading: • Martin Fowler (Patterns of Enterprise Application Architecture)
• Sam Newman (Building Microservices) • Chris Richardson (Microservices.io) • Google SRE Book